

Programmdokumentation

Stand: 17.01.2025

Autoren: Thi Thu Trang Pham, Philipp Mikos, Arne Gudermann

UML-Klassendiagramm

Erstellen Sie ein UML-Diagramm der (Fachkonzept-)Klassen Ihres Spiels in einer **separaten PDF-Datei**. Schwerpunkt des Diagramms sollen die Klassen und ihre Beziehungen untereinander sein. In den Klassen sollen nur die wichtigsten Attribute und Methoden aufgeführt werden.

Hinweise:

- Achten Sie auf eine UML-gerechte Darstellung ihres Diagramms.
- Die Klassen des GameStateManagement-Beispiels müssen nicht dargestellt werden.
- Das Klassendiagramm muss sich ohne Probleme mit lesbarer Schrift auf Papier der maximalen Größe DinA3 ausdrucken lassen!

Unterschiede zum Entwurf

Beschreiben Sie alle inhaltlichen Unterschiede zum UML-Klassendiagramm Ihres Entwurfs und erläutern/begründen Sie diese. Beschreiben Sie neu hinzugekommene Klassen und Schnittstellen in ein bis zwei Sätzen.

Wir haben das Spiel durch neue Waffen und Objekte interessanter gestaltet, wie beispielsweise Feuerbälle, die den Spieler verfolgen können.

Außerdem wurden sechs Gegner hinzugefügt – doppelt so viele wie ursprünglich geplant – sowie einige Hilfsklassen, um die Erstellung und Steuerung der Levels und Räume zu erleichtern. Alle Gegner und Objekte werden dabei von den jeweiligen Räumen gesteuert, statt im Map wie vorher. Dadurch wird die Steuerung der Objekte und Generierung der Räume nach Levels einfacher. Innerhalb eines Levels teilen sich die Räume dieselben Gegnertypen, jedoch in unterschiedlicher Anzahl. Für höhere Level gibt es Gegner mit höherem Niveau.

Das ursprünglich verwendete MDP-Verfahren zur Spielsuche wurde durch den A*-Algorithmus ersetzt. Dies liegt daran, dass das MDP-Verfahren viele Iterationen benötigt, um die Schritte korrekt abzuschätzen, während der A*-Algorithmus einfacher zu implementieren ist und im Spiel effizient funktioniert.

GameEntity erbt nun nicht mehr von DrawableGameComponent weil es sich als schwierig herausgestellt hat wenn Elemente im Spiel automatisch Update() und Draw() ausführen. Beispielsweise ist es dann schwierig das Spiel zu pausieren, da Items und Gegner dann trotzdem noch geupdated werden auch wenn der GameplayScreen seine Elemente nicht mehr updated.

Klasse: Shoes

Diese Klasse erbt von Item und verleiht dem Spieler die Fähigkeit schneller zu laufen.

Klasse: Timer

Diese Klasse ist für das stoppen der Zeit zuständig und benachrichtigt zuhörende GameEntities wenn die Zeit abgelaufen ist.

Klasse: EndDoor

Diese Klasse erbt von Door und stellt eine Tür in das nächste Level dar.

Klasse: Projectile

Diese Abstrakte Klasse ist die Oberklasse für alle Arten von Geschossen (Fireball, Arrow)

Klasse: Arrow

Diese Klasse erbt von Projectile und stellt einen Pfeil dar der vom Bow abgeschossen wird.

Klasse: Plant

Diese Klasse stellt eine Topfpflanze dar. Sie wird nur zur Dekoration verwendet.

Klasse: CheatConsole

Diese Klasse stellt eine Cheatzeile im Spiel dar, die mit "0" geöffnet werden kann. man kann sich mit der eingabe von Zahlen durch die Räume Teleportieren oder sich Gold, Lernpunkte oder Lebenspunkte Cheaten ("gold", "hp", "lp")

Klasse: SoundSettings

Die Klasse SoundSettings enthält die Lautstärkeeinstellungen für Musik und Soundeffekte. Bei Änderungen werden die GameEntities im Spiel benachrichtigt und passen ihre jeweiligen Sounds in der Lautstärke an

Klasse: Trap

Diese Klasse stellt im Spiel eine Bärenfalle da. Der Spieler erleidet Schaden wenn er über die Bärenfalle läuft.

Klasse: Role

Die Klasse Role stellt eine Zauberspruchrolle da und dient als Waffe für den Spieler um Feuerbälle zu werfen. Die Klasse ersetzt die Klasse MagicStick.

Klasse: ParticleSystem

Wir haben eine Klasse ParticleSystem für Visuelle Effekte hinzugefügt.

Klasse: Particle

Diese Klasse stellt ein einzelnes Particle in einem Partikelsystem dar.

Klasse: Enemy_Doubler

Der Enemy_Doubler verfolgt den Spieler mithilfe des A*-Suchverfahrens. Wenn er vom Spieler geschlagen wird, erleidet er Schaden und verdoppelt sich.

Klasse: Enemy_Guard

Der Enemy_Guard bewegt sich zunächst zufällig, bis er sich in der Nähe des Spielers befindet. Sobald das der Fall ist, verfolgt er den Spieler und wirft Feuerbälle auf ihn.

Klasse Enemy_Magie:

Der Enemy_Magie ist für eine Sekunde sichtbar und anschließend für zwei Sekunden unsichtbar. Wenn der Spieler ihn trifft, verliert der Enemy_Magie Lebenspunkte.

Klasse Enemy_prisoner:

Der Enemy_prisoner verfolgt den Spieler mithilfe des A*-Suchverfahrens und bekämpfen wenn er nah zu dem Spieler ist.

Klasse Enemy_Knight:

Der Enemy_Knight sammelt alle Goldstücke in einem Raum, indem er das A*-Suchverfahren verwendet.

Klasse Enemy_Wizard:

Der Enemy_Wizard kombiniert die Fähigkeiten der anderen Gegner. Er verfolgt den Spieler mit dem A*-Algorithmus und wirft gleichzeitig Feuerbälle auf ihn. Außerdem kann er sich unsichtbar machen und sich verdoppeln. Diese Klasse ersetzt die ursprüngliche Klasse EndBoss.

Klasse Fireball:

Der Feuerball dient als Waffe im Spiel. Er kann den Bekämpften verfolgen.

Klasse RoomEnemyConfig:

Die Klasse RoomEnemyConfig wird verwendet, um Konfigurationen für Feinde in einem Raum in dem Spiel zu speichern, damit die dynamische Generierung der Räume einfacher wird.

Einsatz von Entwurfsmustern

Beschreiben Sie, welche Entwurfsmuster Sie final eingesetzt haben. Gibt es Unterschiede zum Entwurf? Hat sich der Einsatz der Entwurfsmuster bewährt? Machen Sie die Entwurfsmuster in Ihrem UML-Klassendiagramm kenntlich.

Muster:

Wir haben letztendlich drei Entwurfsmuster eingesetzt: das Singleton-Muster, das Observer-Muster und das Lazy-Loading-Muster. Auf das Kompositum-Muster haben wir verzichtet.

Singleton-Muster:

Dieses Muster wurde für den Player und den Asset Manager implementiert, um sicherzustellen, dass es in der Anwendung jeweils genau eine Instanz gibt. Es hat sich als nützlich erwiesen, da der Spieler zentral gesteuert wird und globale Zugriffsmöglichkeiten benötigt. Auch auf den AssetManager wird von überall im Code zugegriffen da an verschiedenen Stellen Assets benötigt werden.

Lazy Loading:

Dieses Muster wurde im Asset Manager verwendet, um Texturen erst nachzuladen, wenn sie auch benötigt werden.

Observer-Muster:

Dieses Muster wird verwendet um die Soundeinstellungen zu verwalten. Sobald die Lautstärke vom Benutzer in den Einstellungen verändert wird, sollen alle "zuhörenden" GameEntities die Lautstärke ihrer Sounds anpassen.

Quellenverzeichnis

Führen Sie hier alle Quellen auf, die Sie zur Erstellung der Entwurfsdokumentation genutzt haben.

Eigenständigkeitserklärung

Hiermit erklären wir, dass wir die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben. Alle sinngemäß und wörtlich übernommenen Textstellen aus fremden Quellen wurden kenntlich gemacht.

Rüthen, den 17.01.2025

Unterschrift:



(Arne Gudermann)

Lünen, den 17.01.2025

Unterschrift: 

(Philipp Mikos)

Dortmund, den 17.01.2025

Unterschrift: 

(Thi Thu Trang Pham)